

課題：MIPS CPU シミュレータ

蓬萊 尚幸

【目標】

- ・ MIPS CPU のシミュレータを C 言語で実装する。
- ・ 命令セットすべてを実装しなくてもよい。
- ・ 実際の MIPS CPU はパイプライン実行するが、逐次実行（単一サイクル方式）でよいとする。

【成果物】

- ・ プログラム仕様書 (pdf 形式)
- ・ ソースコード (tar.gz.b64 形式)
- ・ 実行結果 (tar.gz.b64 形式)

1. 全体構成

- ・ 各ステージは引数なし戻り値なしの関数で実装する。
- ・ 汎用レジスタ、専用レジスタ、ステージ間レジスタ、メモリは、大域変数で実装する。
- ・ ステージ間のデータの引継ぎは、すべてステージ間レジスタを介して行う。
- ・ Main 関数は各ステージ用の関数を順次呼び出す無限ループで実装する。
- ・ 命令解釈ステージで不明な命令を検知したときには異常終了する。
- ・ BREAK 命令(0x0000000C)を検知したときには正常終了する。
- ・ コード列のセット、シミュレーションの実行制御、レジスタやメモリの内容確認を行う機能（シミュレーション機能）を用意する。

【設計ポイント】

- 各ステージ用の関数のアルゴリズム。
- 命令形式(R,I,J)の違いを意識した構造化。
- シミュレーション機能。

2. データ構造

2.1 レジスタ

- ・ 汎用レジスタと専用レジスタはすべて 32 ビットなので、`uint32_t` 型で実装する。
- ・ 各ステージ間レジスタは、ハードウェア実装ではすべて 32 ビットであるが、本シミュ

レーションでは、それぞれ、意味を反映した型のメンバで構成される **struct** 型にまとめ、変数 1 個で実装する。

【設計ポイント】

- ・ 各ステージ間レジスタに対応した **struct** 型。

2.2 メモリ

MIPS アーキテクチャではビッグエンディアンが採用されているが、実習で用いる PC(インテル CPU)ではリトルエンディアンである。また、メモリはバイト単位でアドレスが付与されているが、MIPS アーキテクチャではワードデータやハーフワードデータについて境界をアラインメントして使用する。そこで、シミュレータの実装を容易にするために、アラインメント違反を検知するビッグエンディアンであるメモリを仮想的に実現するモジュール MemoryModule を提供する。

3. 命令

- ・ **uint32_t** 型で実装する。
例 : `add $t0,$s1,$s2 = opcode(add):0, rs($s1)=1610, rt($s2)=1710, rd($t0)=810, shmt(無)=0, funct(add)=2016 ⇒ 000000 10000 10001 01000 00000 0101002`
- ・ 必ず実装する命令 : `lw, sw, add, addi, sll, nor, slt, srl, beq, j` の 10 個。
- ・ 以下の命令の実装にもチャレンジしてください。
 - その他の基本命令 : 20 個。
 - 乗除算命令 : 6 個、専用レジスタ Hi, Lo の追加が必要。
 - 浮動小数点数命令 : 20 個、浮動点数レジスタとレジスタ FPcond の追加が必要。
命令形式 FR, FI の増加に注意。

4. とても難しい課題

- ・ さらなるチャレンジとしては、以下の項目があります。
 - 排他制御用の `ll` 命令、`sc` 命令 : `ll` と `sc` の実行の間で、他の CPU からのメモリアクセスが有った場合となかった場合をシミュレートできるようにする。
 - 例外処理用の `mfc0` 命令 : CP0 のレジスタを追加し、例外をシミュレートできるようにする。

以 上

